Machine Translated by Google



Tetris rouge

Tetris Network avec sauce Red Pelicans

Résumé : L'objectif de ce projet est de développer un jeu Tetris multijoueur en réseau utilisant une pile logicielle exclusivement composée de JavaScript Full Stack.

Version : 5.2

Machine Translated by Google

Contenu

je Avant-propos 2

II Introduction 3

III Objectifs 4

IV Instructions générales 5

V Partie obligatoire V.1 6
Tetris : Le Jeu 6
V.1.1 Mouvement de la pièce 7
V.2 Tetris : Détails techniques 7
V.2.1 Gestion du jeu 8
Configuration du serveur V.2.2 8
Configuration du client V.2.3 9
V.2.4 Modèle standard 10
Tests V.2.5 10

VI Fête de bonus 11

VII. Soumission et évaluation par les pairs 12

Machine Translated by Google

Chapitre I

Avant-propos

Redpelicans est le sponsor de ce projet. Vous comprenez la tendance fortement mise en évidence en rouge dans le jeu Tetris : nous vous proposons de construire et de faire voler des pélicans sur vos terrains de jeu, au cœur d'une forêt de tétriminos.

Machine Translated by Google

Chapitre II

Introduction

Tout le monde connaît le jeu Tetris, et tout le monde connaît JavaScript. Il ne reste plus qu'à créer un jeu Tetris en JavaScript.
Oui, mais...
Votre partie de Tetris sera en ligne et multijoueur. Vous pourrez ainsi interrompre vos sessions de jeu intergalactiques pendant vos marathons de programmation nocturnes. (Il subsiste quelques problèmes de Wi-Fi sur certaines planètes.)

Votre jeu Tetris exploitera des technologies JavaScript de pointe, qui sont au cœur d'une féroce concurrence intellectuelle, industrielle et financière entre Facebook et Google dans leur quête de domination mondiale.

Concevoir votre Tetris exigera de la réflexion : vous devrez architecturer l'application, définir un protocole réseau asynchrone, l'implémenter en utilisant les principes de la programmation fonctionnelle, et animer et afficher le tout en utilisant HTML !

Bonne chance, bon codage… et n’oubliez pas de tester — et de tester encore !

Machine Translated by Google

Chapitre III

Objectifs

Les objectifs pédagogiques sont multiples. Le but principal est de présenter le langage JavaScript, d'explorer son riche écosystème et de mettre en œuvre plusieurs principes, techniques et outils phares du développement Full Stack JavaScript.

Tout le monde prétend connaître JavaScript, mais très peu maîtrisent réellement ce langage aux multiples facettes — qui est partiellement fonctionnel, entièrement basé sur des prototypes, à typage dynamique, résolument asynchrone et étonnamment efficace.

En créant un jeu Tetris en réseau, vous pourrez :

- Appliquer les principes de la programmation fonctionnelle (une exigence).
- Développer une logique client et serveur asynchrone (de par la nature de JavaScript).
- Mettre en œuvre des modèles réactifs (de par la nature du jeu et de l'interface utilisateur).

Vous devrez également rédiger des tests unitaires conformes aux normes industrielles pour les pipelines de livraison continue.

Machine Translated by Google

Chapitre IV

Instructions générales

Le projet doit être entièrement développé en JavaScript, en utilisant la dernière version disponible.



TypeScript est autorisé, car il s'agit d'un sur-ensemble strict de JavaScript et il est compilé en JavaScript standard.

Le code côté client (navigateur) doit être écrit sans utiliser le mot-clé `this`, afin de privilégier les constructions fonctionnelles aux constructions orientées objet. Vous pouvez choisir une bibliothèque fonctionnelle comme lodash ou ramda, ou n'en utiliser aucune.

La logique du jeu gérant le plateau et les pièces doit être implémentée à l'aide de fonctions pures.
La seule exception à cette règle : ceci peut être utilisé pour définir des sous-classes personnalisées d'Error.

À l'inverse, le code côté serveur doit adopter une approche orientée objet utilisant des prototypes. Il convient au minimum de définir des classes pour Joueur, Pièce et Jeu.

L'application cliente doit être développée avec un framework JavaScript moderne. Le HTML ne doit pas utiliser d'éléments <TABLE />. La mise en page doit utiliser Grid ou Flexbox.

Les éléments suivants sont interdits :

- Bibliothèques de manipulation du DOM (par exemple, jQuery).
- Toile.
- SVG (Scalable Vector Graphics).

Il n'est pas nécessaire de manipuler directement le DOM.

Les tests unitaires doivent couvrir au moins 70 % des instructions, des fonctions et des lignes, et au moins 50 % des branches.

Machine Translated by Google

Chapitre V

Partie obligatoire

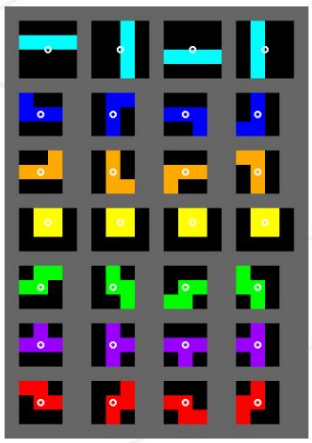
Tetris version 1 : Le Jeu

Tetris est un jeu de puzzle (voir Wikipédia) dont le but est d'aligner des pièces qui tombent sur la grille le plus longtemps possible. La partie se termine lorsqu'il n'est plus possible d'ajouter de nouvelles pièces. Compléter une ligne la fait disparaître, prolongeant ainsi la partie.

Votre version conservera les mécanismes de base, mais permettra le multijoueur. Chaque joueur disposera de son propre plateau et recevra la même séquence de pièces. Lorsqu'un joueur élimine des lignes, ses adversaires recevront n - 1 lignes de pénalité indestructibles en bas de leur plateau.

Chaque champ comporte 10 colonnes et 20 lignes. Les joueurs peuvent voir le nom de leurs adversaires ainsi qu'une représentation graphique de leur champ. Cette représentation indique la hauteur du bloc le plus haut de chaque colonne. Elle est mise à jour en temps réel.

Le jeu utilisera les formes et les règles de rotation originales du Tetrimino :



Machine Translated by Google

Il n'y a pas de système de points ; le dernier joueur en lice remporte la partie. Le jeu propose des modes solo et multijoueur.

Mouvement de la pièce V.1.1

Les pièces tombent à vitesse constante. Dès qu'une pièce touche la pile (les pièces déjà présentes), elle ne s'immobilise que sur l'image suivante, ce qui permet des ajustements de dernière minute.

Entrées disponibles pour le joueur :

Flèches gauche/droite : Déplacer la pièce horizontalement.

Flèche vers le haut : Faire pivoter la pièce.

Flèche vers le bas : Chute douce.

Barre d'espace : Atterrissage brutal pour combler un espace.

Tetris V.2 : Détails techniques

L'architecture du jeu repose sur un modèle client/serveur. Les clients s'exécutent dans les navigateurs modernes, tandis que le serveur fonctionne sous Node.js. La communication utilise HTTP et socket.io pour les échanges d'événements bidirectionnels.

Le serveur gère :

- Gestion des jeux et des joueurs
- Distribution des pièces
- Mises à jour du spectre



Dans une même partie, chaque joueur doit recevoir les mêmes pièces aux mêmes positions et coordonnées, même si ce n'est pas à des moments précis.

Aucune persistance des données n'est nécessaire.

L'utilisation de composants fonctionnels (par exemple, les Hooks React) est encouragée .

Le client doit être une application monopage.

Machine Translated by Google

V.2.1 Gestion du jeu

Les joueurs rejoignent les parties via une URL du type :

- http://<nom_serveur_ou_adresse_ip>:<port>/<salle>/<nom_joueur>.

salle : Nom du jeu auquel rejoindre.

nom_du_joueur : Nom du joueur.



Utilisez des routeurs appropriés tels que BrowserRouter ou MemoryRouter pour la gestion des URL.

Le premier joueur à rejoindre la partie devient l'hôte et décide quand démarrer ou redémarrer la partie.
Si l'hôte actuel quitte la partie, un des joueurs restants prendra sa place. Une fois la partie commencée, aucun nouveau joueur ne peut la rejoindre avant la manche suivante.

La partie se termine lorsqu'il ne reste plus qu'un seul joueur.

On peut jouer à ce jeu avec un seul joueur.

Plusieurs jeux simultanés sont pris en charge.

Configuration du serveur V.2.2

Le serveur gère la logique du jeu et les communications. Identifiez et définissez clairement les responsabilités partagées entre les clients et le serveur, et spécifiez le protocole réseau.

Le serveur exécute une boucle asynchrone gérant les événements via socket.io. Il doit également servir index.html, bundle.js et toutes les ressources statiques via HTTP.

Machine Translated by Google

Configuration du client V.2.3

Le client est une application monopage (SPA) basée sur un navigateur :

- Le navigateur charge index.html, qui contient une référence à bundle.js contenant l'application complète.
- Aucun autre fichier HTML n'est échangé. Le rendu et la logique sont entièrement gérés côté client.
  côté.
- La communication avec le serveur se fait via socket.io.

Nous recommandons d'utiliser :

- Un framework JS comme React ou Vue pour la couche de présentation (MVC)
- Redux pour gérer l'état de l'application.

Vous pouvez améliorer votre application en utilisant des packages de npm :

- Fonctionnel : lodash, Ramda (facultatif)
- Immuabilité : Immutable.js, ou utiliser les fonctionnalités ES (par exemple, la dispersion des objets)
- Asynchrone : Redux est synchrone par défaut ; utilisez redux-thunk ou redux-promise pour les flux de travail asynchrones



Évitez d'utiliser des bibliothèques obsolètes. Si nécessaire, recherchez des bibliothèques modernes.
alternatives.

Machine Translated by Google

V.2.4 Modèle standard

Pour vous éviter une configuration fastidieuse, nous fournissons un modèle de configuration :

- Lancez le serveur.
- Générer des bundles JS pour le navigateur.
- Exécuter les tests unitaires et de couverture.

Dépôt GitHub : red_tetris_boilerplate
Documentation : README

Tests V.2.5

Les tests sont utiles :

- Améliorer la fiabilité des versions.
- Accélérer la livraison grâce à l'automatisation.
- Garantir la qualité du produit et sa maintenabilité à long terme.

JavaScript est désormais adapté aux environnements d'entreprise. À l'instar de .NET ou Java par le passé, il constitue le socle de « JavaScript d'entreprise ». Les pipelines de tests en sont un élément central, garantissant la détection automatique des versions défectueuses.

Les tests unitaires doivent couvrir au moins 70 % des instructions, des fonctions et des lignes, et 50 % des branches.

Plus précisément, lors de l'exécution des tests, vous obtiendrez 4 indicateurs :

- Déclarations : taux de couverture des déclarations
- Fonctions : taux de couverture des fonctions
- Lignes : taux de couverture des lignes de code
- Branches : taux de couverture des chemins d'exécution du code

plaque standard comprend un pipeline de test et des exemples de tests (voir la documentation).



Pour des raisons évidentes de sécurité, ne stockez en aucun cas les identifiants, les clés API ou les variables d'environnement dans le dépôt. Utilisez un fichier `.env` et assurez-vous qu'il soit ignoré par Git. La divulgation de données sensibles entraînera l'échec du projet.

Machine Translated by Google

Chapitre VI

Partie bonus

Red_Tetris est un jeu vidéo, il existe donc de nombreuses possibilités d'ajouter des éléments supplémentaires.

Voici quelques suggestions :

- Ajouter un système de notation.
- Conserver les scores des joueurs.
- Introduire de nouveaux modes de jeu (par exemple, des pièces invisibles, une gravité accrue).

Ce projet propose par défaut l'utilisation de React. Toutefois, si vous souhaitez explorer des concepts plus avancés, nous vous encourageons à vous intéresser à la programmation réactive fonctionnelle. (FRP), un paradigme passionnant particulièrement bien adapté à ce type de projet.
Découvrez flyd, Une bibliothèque minimaliste dotée d'une API attrayante.



Les fonctionnalités bonus ne seront prises en compte que si les conditions obligatoires sont remplies et fonctionnelles. Dans le cas contraire, elles ne seront pas évaluées.

Machine Translated by Google

Chapitre VII

Soumission et évaluation par les pairs

Soumettez votre projet dans votre dépôt Git comme d'habitude. Seul le contenu de votre dépôt sera évalué. Vérifiez attentivement vos dossiers et noms de fichiers pour vous assurer que tout est correctement nommé.

Le jeu doit être parfaitement fonctionnel.